

JAI ME GAMERO MATOS DOCUMENTACIÓN LIBRERÍA

IES Arroyo Harnina



Introducción

Vamos a tratar de documentar el proyecto dividiéndolo por los paquetes y secciones en los que he organizado el proyecto respetando el patrón de arquitectura modelo vista controlador.

Paquete modelo

El paquete modelo es el que contendrá toda la lógica.

Clase Libro

Variables:

- String titulo
- String autor
- String isbn
- String editorial

-
- String formato
 - String estado
 - int stock

Métodos:

- Constructor: En este caso el constructor inicializa todas las variables de manera estándar, las iguala al mismo parámetro que se le pasa al método.
- Getters & Setters: Estos métodos modifican o devuelven los valores de las variables que contiene la clase Libro.
- ToString: Devuelve String con todos los valores que se le han asignado al libro al instanciarlo.

Clase Libreria

Variables:

- ArrayList<Libro>

Métodos:

- Constructor: En este caso aceptaría meterle una lista por parámetros, que de hecho la vamos a meter en el constructor de la clase eventUI pero a esta lista le asignaremos el valor que retorne el método “establecerListaDeLibroPorDefecto()”.
- Getters & Setters: Modifican o retornan la variable arrayList<Libro> que contiene la Libreria.
- establecerListaDeLibroPorDefecto: no acepta nada por parametros ya que su función es crear y retornar una lista y agregarle una serie de libros para que cuando se instancie la vista ya haya libros en la tabla.
- obtenerLibro: esta función recibe por parámetros un String que para utilizarlo como criterio de búsqueda comparándolo mientras que se recorre la lista para compararlo con los ISBN de los libros que contiene la lista de la librería, si encuentra una coincidencia, retorna un String con los valores del libro generados por el método ToString de la clase Libro.
- obtenerLibroDos: A diferencia del primero este puede retornar un valor nulo.
- comprobarIsbnExistente: Acepta por parámetros un String para compararlo con los ISBN que contiene la lista recorriendola y si lo encuentra devuelve un resultado booleano verdadero y en caso de no encontrarlo lo retorna como falso.
- sumarAlStockPorIsbn: Recibe por parámetros el String del ISBN y cuando encuentra la coincidencia le suma uno al stock con el que coincide en ISBN para evitar duplicidades.

-
- obtenerIdSeleccionado: Recibe por parámetros un componente JTable que es un componente de la vista y cuando detecte que el usuario a lo ha seleccionado pulsando sobre él en la tabla te devuelve el número de la posición de la fila que has seleccionado.

Paquete validaciones

Este apartado será el encargado de la lógica detrás de la comprobación de que la comprobación de los datos introducidos sea la correcta cumpliendo así con los requisitos del programa.

Clase Validaciones

Variables:

No contiene parámetros ya que lo que buscamos es su pura utilidad en forma de métodos y como forma de agrupar todas las validaciones en una sola clase.

Métodos:

- validarLetras: Recibe un String y evalúa si sigue un patrón que a su vez recibe un regex, es decir a través de este regex que lo que hace es evaluar si está dentro de todos los caracteres del alfabeto español y lo compara con la variable que ha recibido para ver si lo cumple retornando un booleano.
- esUnNumero: Recibe un String y evalúa si sigue el patrón que en este caso es un regex que evalúa si todos sus caracteres son un número entre el 0 y el 9 devolviendo un booleano.
- validarISBN: Recibe un ISBN en forma de String por parámetros y evalúa si su longitud es igual a trece y si es un número utilizando el método esUnNumero devolviendo un booleano con el resultado.
- esUnFloat: Recibe por parámetros un String que transforma parseandolo a float y evaluando si el último carácter es una f o una d ya que rompería el programa y lo evalúa devolviendo un booleano.

Paquete imagenes

Este paquete está dedicado a contener todas las imágenes que va a tener la aplicación que en este caso solo contiene una de un libro a la cual se le hará referencia en la vista.

Paquete vista

Este paquete se encargará de gestionar todo lo relacionado con la vista o interface de usuario que es el intermediario entre el usuario y la lógica del programa.

Interface AccesoUI

Esta interface contiene los métodos que entregan los componentes de la vista para conectarlos con la lógica en el controlador en este caso no están declaradas aún obviamente solo especifican el tipo que retornan.

Métodos:

- botonGuardar, botonBorrar, botonSalir, botonConsultar, botonModificar: Retornan un objeto de tipo botón.
- campotxtPrecio, campotxtISBN, campotxtTitulo, campotxtEditorial, campotxtAutor: Retornan un objeto de tipo campo de texto.
- tablaEstanteria: Retorna un objeto de tipo JTable.
- rdbtnCartone, rdbtnRustica, rdbtnGrapada, rdbtnEspiral, rdbtnReedicion, rdbtnNovedad: Retornan un objeto de tipo JRadioButton, que son los botones de opción con los que elegimos el formato y el estado del libro.
- grupoFormato, grupoEstado: Retornan un objeto de tipo ButtonGroup, que es el que agrupa los radio buttons para que solo se pueda marcar uno de cada grupo a la vez. Los necesitamos en el controlador para poder dejarlos sin seleccionar cuando se guarda un libro.

Clase UI

Esta clase es la vista como tal. Hereda de JFrame, que es la ventana de la aplicación, e implementa la interface AccesoUI, así que está obligada a definir todos los métodos que hemos visto antes. Aquí está construida toda la parte visual y no hay ninguna lógica del programa, solo se crean los componentes y se les da el aspecto que queremos.

Variables:

- Constantes de diseño: RUTA_IMAGEN con la ruta de la imagen del libro, los colores (COLOR_FONDO, COLOR_SUPERFICIE, COLOR_PRIMARIO, COLOR_ACENTO, COLOR_PELIGRO, COLOR_TEXTO, COLOR_TEXTO_SUAVE, COLOR_BORDE, COLOR_FILA_ALTERNA y COLOR_SELECCION) y las fuentes (FUENTE_TITULO, FUENTE_SECCION, FUENTE_BASE y FUENTE_NEGRITA). Las he dejado todas juntas arriba para que si queremos cambiar el aspecto de la aplicación solo haya que tocarlas en un sitio.
- contentPane: Es el JPanel principal de la ventana donde se van colocando el resto de paneles.
- botonGuardar, botonSalir, botonBorrar, botonConsultar, botonModificar: Son los JButton de la aplicación.
- campoTxtPrecio, campoTxtISBN, campoTxtTitulo, campoTxtEditorial, campoTxtAutor: Son los JTextField donde el usuario escribe los datos del libro.
- rdbtnCartone, rdbtnRustica, rdbtnGrapada, rdbtnEspiral, rdbtnReedicion, rdbtnNovedad: Son los JRadioButton del formato y del estado.
- grupoFormato, grupoEstado: Los ButtonGroup a los que se añaden los radio buttons.
- tablaEstanteria: La JTable donde se muestran los libros de la librería.

Métodos:

- Constructor: Es el que monta toda la ventana. Primero configura lo básico, que al cerrarla se termine el programa, el tamaño, un tamaño mínimo y que salga centrada en la pantalla. Después le pone al panel principal un BorderLayout y reparte tres zonas. Arriba va la cabecera con el título “LIBRERÍA JAIME” sobre fondo morado y una línea naranja debajo, abajo la barra de botones con SALIR a la izquierda y CONSULTAR, MODIFICAR, BORRAR y GUARDAR a la derecha, y en el centro un JTabbedPane con dos pestañas. La pestaña LIBRO tiene dos tarjetas, una llamada DATOS DEL LIBRO con las etiquetas, los campos de texto y la imagen, y otra llamada CARACTERÍSTICAS con los radio buttons de formato y estado. La pestaña ESTANTERIA contiene la tabla dentro de un JScrollPane, y en ella se sobrescribe el método prepareRenderer para que las filas se pinten alternando dos colores y sea más fácil leerlas.
- cargarImagen: No recibe nada y retorna un Icon. Intenta buscar la imagen primero dentro del classpath y si no la encuentra la busca como archivo normal con la ruta que le hemos puesto. Si no la encuentra o no se puede leer, en lugar de dejar un hueco vacío retorna un IconoLibros, que es una imagen dibujada por código. Si sí la encuentra la escala para que quepa en un cuadro de 170 por 160 sin agrandarla nunca.
- estilizarEtiqueta: Recibe un JLabel y le asigna la fuente, el color y un ancho fijo para que todas las etiquetas queden alineadas.
- estilizarCampo: Recibe un JTextField y le da la fuente, los colores y un borde redondeado. Además le añade un FocusListener para que cuando el campo gane el foco el borde cambie a morado y más grueso, y cuando lo pierda vuelva al borde normal.

-
- `estilizarRadio`: Recibe un `JRadioButton` y le quita el fondo, le asigna fuente y color y hace que el cursor cambie a una mano al pasar por encima.
 - `estilizarTabla`: Recibe un `JTable` y configura la altura de las filas, quita las líneas verticales, asigna los colores de selección y le da color y fuente a la cabecera de la tabla.
 - Métodos de `AccesoUI`: `botonGuardar`, `botonBorrar`, `botonSalir`, `botonConsultar`, `botonModificar`, `campoTxtPrecio`, `campoTxtISBN`, `campoTxtTitulo`, `campoTxtEditorial`, `campoTxtAutor`, `tablaEstanteria`, `rdbtnCartone`, `rdbtnRustica`, `rdbtnGrapada`, `rdbtnEspiral`, `rdbtnReedicion`, `rdbtnNovedad`, `grupoFormato` y `grupoEstado`. Aquí ya sí están definidos, llevan `@Override` porque vienen de la interface y lo único que hacen es retornar el componente que les corresponde, que es lo que luego usa el controlador.

Clases internas:

Dentro de la clase `UI` he creado tres clases privadas y estáticas que sirven solo para personalizar el aspecto de algunos componentes.

- `IconoLibros`: Implementa la interface `Icon`. Recibe un ancho y un alto por el constructor y con `getIconWidth` y `getIconHeight` los devuelve. En `paintIcon` utiliza `Graphics2D` para dibujar una pila de cuatro libros de colores con una pequeña sombra debajo. Es la imagen de repuesto por si no se carga la del paquete `imagenes`.
- `BotonModerno`: Hereda de `JButton` y es el botón redondeado que usamos en toda la aplicación. En el constructor recibe el texto, el color de fondo, el del texto, el del borde y un booleano que indica si el botón va relleno o solo con borde. Le añade un `MouseListener` que detecta cuando el ratón entra o sale para cambiar el color al pasar por encima, y en `paintComponent`, que está sobrescrito, es donde se pinta el fondo y el borde, oscureciendo el color si el botón está pulsado.
- `PestanasModernas`: Hereda de `BasicTabbedPaneUI` y cambia cómo se dibujan las pestañas. Sobrescribe los métodos para que la pestaña seleccionada tenga fondo blanco con una línea naranja arriba, y elimina el borde del contenido y el recuadro de foco que salen por defecto.

Paquete controlador

Este paquete es el que une la vista con el modelo. Es donde se escuchan las acciones del usuario, como pulsar un botón, y se decide qué hay que hacer con la lógica del programa y cómo se actualiza lo que se ve en pantalla.

Clase EventUI

Es la clase que contiene todos los eventos de la aplicación. Recibe la vista a través de la interface AccesoUI y no directamente la clase UI, de esta forma el controlador no depende de cómo esté hecha la ventana, solo de los componentes que le entrega la interface.

Variables:

- AccesoUI UI: La vista a la que accede el controlador para coger los componentes.
- Libreria libreria: La librería con la lista de libros sobre la que se hacen todas las operaciones.

Métodos:

- Constructor: Recibe la vista por parámetros y la guarda, crea la Libreria (le pasamos null porque la lista de libros por defecto se genera desde la propia clase Libreria), llama al método instanciarMetodos para activar todos los botones y por último llama a rellenarTabla para que al abrir la aplicación ya salgan los libros en la estantería.
- instanciarMetodos: No recibe nada, simplemente llama uno por uno a los métodos click para dejar todos los botones funcionando.
- clickSalir: Añade un ActionListener al botón de salir que muestra un cuadro de confirmación con JOptionPane preguntando si estamos seguros, y si la respuesta es sí cierra el programa con System.exit(0).
- clickGuardar: Añade un ActionListener al botón de guardar. Lo primero que hace es comprobar los campos con aplicarTodasLasValidaciones y si no son correctos muestra el mensaje “Campos erróneos”. Si lo son crea el libro con crearLibro y mira con comprobarIsbnExistente si ya hay uno con ese ISBN. Si no existe lo añade a la librería, actualiza la tabla, limpia los campos y los radio buttons y avisa de que se ha guardado. Si ya existía no lo duplica, sino que le suma uno al stock con sumarAlStockPorIsbn y avisa de ello.
- clickBorrar: Añade un ActionListener al botón de borrar. Obtiene la fila seleccionada con obtenerIdSeleccionado y si devuelve -1 es que no hay ninguna, así que muestra un aviso y termina. Si hay un libro seleccionado y su stock es mayor que uno solo le resta una unidad, y si solo quedaba una lo borra por completo de la lista con borrarLibros. En ambos casos actualiza la tabla y muestra un mensaje.
- clickConsultar: Añade un ActionListener al botón de consultar que pide un ISBN con un cuadro de diálogo y, si existe en la librería, muestra los datos del libro con el método obtenerLibro.
- clickModificar: Añade un ActionListener al botón de modificar. Igual que al borrar, primero comprueba que haya una fila seleccionada y si no avisa. Después va pidiendo un nuevo valor para cada dato (ISBN, título, editorial, autor, precio y stock) con el método pedirValidado, cada uno con su validación. En el caso del ISBN además de ser válido no puede coincidir con el de otro libro distinto, aunque sí puede dejar el mismo que ya tenía. Al terminar actualiza el libro con los setters, recarga la tabla y muestra un mensaje de que se ha modificado.

-
- pedirValidado: Recibe el nombre del campo, el valor actual y un Predicate<String> que es la validación que tiene que cumplir. Muestra un cuadro de diálogo con el valor actual ya escrito y si el usuario cancela o lo deja vacío retorna el valor que ya tenía. Si escribe algo que cumple la validación lo retorna, y si no la cumple muestra un mensaje de error y vuelve a preguntar, repitiéndose dentro de un bucle hasta que el valor sea correcto.
 - rellenarTabla: Recibe el JTable de la vista. Crea un array con los nombres de las columnas y una matriz de Strings en la que recorre la lista de libros y va metiendo en cada fila el ISBN, título, editorial, autor, precio y stock. Con eso crea un DefaultTableModel y se lo asigna a la tabla con setModel, de forma que cada vez que se llama la tabla se refresca con los datos actuales.
 - getRadioButtonFormato: Comprueba uno por uno qué radio button de formato está seleccionado y retorna su texto. Si no hay ninguno retorna null.
 - getRadioButtonEstado: Hace lo mismo que el anterior pero con los radio buttons de estado, Novedad y Reedición.
 - limpiarRadioButtons: Deja sin seleccionar los dos grupos de radio buttons utilizando clearSelection.
 - aplicarTodasLasValidaciones: Comprueba con la clase Validaciones que el ISBN sea correcto, que autor, editorial y título solo tengan letras y que el precio sea un float. Retorna verdadero solo si se cumplen todas a la vez.
 - crearLibro: Retorna un objeto Libro creado con lo que hay escrito en los campos de texto, el formato y el estado de los radio buttons, el precio convertido a float y un stock inicial de 1.
 - limpiarCamposLibro: Deja vacíos todos los campos de texto de la vista.

Clase Main

Es la clase con la que arranca la aplicación, está dentro del paquete controlador y no tiene variables, sólo contiene el método main.

Métodos:

- main: Lanza la aplicación dentro de un EventQueue.invokeLater con un Runnable, que sirve para que la ventana se cree en el hilo de eventos de Swing, que es como se recomienda hacerlo. Dentro crea la vista con new UI(), crea el controlador EventUI pasándole esa vista para que conecte los botones con la lógica y por último hace visible la ventana con setVisible(true). Todo va dentro de un try catch que imprime el error por consola si algo falla al arrancar.